New upper bounds on covering codes $K_q(n, R)$ for alphabets of size six and seven

Mark Marosi* and Claude (Anthropic)†

August 2026

Abstract

We present improved upper bounds for nine entries of the standard tables of bounds on $K_q(n, R)$, the minimum cardinality of a q-ary code of length n with covering radius R, for $q \in \{6, 7\}$: $K_6(7, 3) \le 232$, $K_6(8, 3) \le 1045$, $K_6(8, 4) \le 167$, $K_6(9, 4) \le 703$, $K_6(9, 5) \le 123$, $K_6(10, 4) \le 2951$, $K_6(10, 5) \le 610$, $K_7(8, 4) \le 329$, and $K_7(9, 4) \le 1743$. The previous best bounds, recorded in Kéri's tables (last updated 2011), all arose from general constructions (direct sums and related product rules) rather than from explicit search; to our knowledge these are the first improvements to any upper bound on $K_q(n, R)$ with $q \ge 5$ since 2011. The new bounds were found by focused local search seeded with the construction-based incumbents. All nine codes are given explicitly in the ancillary files, together with a standalone verifier; each code was checked by four independent exhaustive verification methods.

1 Introduction

Let $\mathbb{Z}_q^n$ denote the Hamming space of q-ary words of length n. A code $C \subseteq \mathbb{Z}_q^n$ has *covering radius* R if every word of $\mathbb{Z}_q^n$ is within Hamming distance R of some codeword. The function

$$K_q(n, R) \;=\; \min\{\, |C| : C \subseteq \mathbb{Z}_q^n \text{ has covering radius} \le R \,\}$$

is a central quantity of covering-code theory [1]; the binary case $K_2(n, 1)$ includes the domination number of the hypercube, and the ternary case $K_3(n, 1)$ is the classical football pool problem.

Upper bounds on $K_q(n, R)$ are established by explicit constructions, and the state of the art is recorded in the tables of Kéri [2], which subsume and extend the earlier tables of Cohen, Honkala, Litsyn and Lobstein [1]. The last update to Kéri's tables was made in November 2011. Recent activity on $K_q(n, R)$ has concentrated on *lower* bounds: Wu and Chen [4] improved binary lower bounds via congruence arguments, and Gijswijt and Polak [3] obtained many new lower bounds for $q \le 5$ from semidefinite programming. On the upper-bound side, for alphabets $q \ge 5$ we are not aware of any published improvement since the 2011 freeze; a systematic search of the literature (arXiv, DBLP, OpenAlex, Lobstein's covering-code bibliography, and the publication lists of the authors active in this area) found none.

*Department of Measurement and Information Systems, Budapest University of Technology and Economics (BME), Budapest, Hungary. email: marosi@mit.bme.hu.

†The search software, the verification protocol, the constructions, and this manuscript were produced autonomously by the AI system Claude (Anthropic) on a single NVIDIA GH200 node; M.M.'s contribution was the idea of applying large-scale GPU/CPU search to the frozen covering-code tables and providing the computational resources. See Section 3.

The reason the entries treated here were improvable is visible in Kéri's own source annotations. For moderate q and $R \geq 3$ nearly all tabulated upper bounds carry keys denoting *general constructions*: the direct sum

$$K_q(n_1 + n_2,\, R_1 + R_2) \;\leq\; K_q(n_1, R_1) \cdot K_q(n_2, R_2), \tag{1}$$

the special case $K_q(n+1, R) \leq q\, K_q(n, R)$, the monotonicity $K_q(n+1, R+1) \leq K_q(n, R)$, and product rules over alphabet factorizations. These are inherited arithmetic: no search was ever run in those cells. The present note shows that plain local search, seeded with the construction that produced the incumbent, beats nine such bounds in the range $q \in \{6, 7\}$, $n \leq 10$ — in one case by 23%.

2 Results

cell	previous bounds	new UB	Δ	code file	previous UB from
$K_6(7,3)$	70–246	**232**	-14	`K6_7_3_M232.txt`	$6 \cdot K_6(6,3)$
$K_6(8,3)$	246–1080	**1045**	-35	`K6_8_3_M1045.txt`	$K_6(4,1) \cdot K_6(4,2)$
$K_6(8,4)$	46–216	**167**	-49	`K6_8_4_M167.txt`	$6 \cdot K_6(7,4)$
$K_6(9,4)$	136–738	**703**	-35	`K6_9_4_M703.txt`	$K_6(3,1) \cdot K_6(6,3)$
$K_6(9,5)$	32–144	**123**	-21	`K6_9_5_M123.txt`	alphabet product rule
$K_6(10,4)$	417–2952	**2951**	-1	`K6_10_4_M2951.txt`	$K_6(4,1) \cdot K_6(6,3)$
$K_6(10,5)$	83–615	**610**	-5	`K6_10_5_M610.txt`	$K_6(6,3) \cdot K_6(4,2)$
$K_7(8,4)$	76–343	**329**	-14	`K7_8_4_M329.txt`	$K_7(7,3)$ (monotonicity)
$K_7(9,4)$	264–1843	**1743**	-100	`K7_9_4_M1743.txt`	direct sum

Table 1: New upper bounds. "Previous bounds" are the lower–upper bound pairs of Kéri's tables [2]; lower bounds are unaffected by this work. The last column gives the construction from which the previous upper bound was inherited, per the source keys of [2].

The nine codes are supplied as ancillary files (one codeword per line, digits over $\{0, \ldots, q-1\}$). None of the searches had converged when stopped, so we expect several of these bounds to be improvable further by the same method with more computation.

Remark 1. *An audit of the tables of [2] against their own derivation rules found one internal gap: the tabulated* $K_{17}(7,2) \leq 252735$ *is weaker than* $K_{17}(7,2) \leq 17 \cdot K_{17}(6,2) = 17 \cdot 14424 = 245208$*, which follows from the tabulated* $K_{17}(6,2) \leq 14424$ *and the standard rule* $K_q(n{+}1, R) \leq q\, K_q(n, R)$*. We record this as an observation; it involves no new construction.*

3 Method

3.1 Search

We search for a code of prescribed size M minimizing the number of uncovered words. The state is a multiset of M codewords together with the coverage count $\mathrm{cnt}(w)$ of every $w \in \mathbb{Z}_q^n$; a move changes one coordinate of one codeword. The move update is cheap because of the following observation.

Lemma 1. *Let c' be obtained from c by setting coordinate p to $v \neq c_p$. Then the set of words leaving the radius-R ball of c and the set of words entering the ball of c' are both indexed by the words at distance exactly R from c in the coordinates other than p; consequently a move updates exactly $2S$ coverage counters with $S = \binom{n-1}{R}(q-1)^R$, the two sets are disjoint, and distinct counter updates address distinct words.*

Proof. Since $d(w, c') = d(w, c) + [\, w_p = c_p \,] - [\, w_p = v \,]$, a word can leave the ball of c only if $d(w, c) = R$ and $w_p = c_p$, and can enter the ball of c' only if $d(w, c') = R$ and $w_p = v$; writing such a word as its restriction to the coordinates $\neq p$ (at distance exactly R from the restriction of c) plus its value at p gives the stated bijections, disjointness ($w_p = c_p$ versus $w_p = v$), and injectivity. □

The search itself is a focused local search in the WalkSAT lineage, close in spirit to the tabu searches that produced much of the original tables [6, 5]: pick a random uncovered word u; collect as candidate moves every codeword at distance exactly $R + 1$ from u nudged one coordinate towards u (if no codeword is that close, the codewords at minimum distance from u are used); evaluate all candidates exactly and in parallel; commit a best candidate, ties broken uniformly at random. On stagnation a codeword is teleported onto an uncovered word. Parameter studies on $K_6(6, 3)$ showed that *candidate breadth dominates*: evaluating all candidate moves exactly outperformed sampling 24 or 64 of them (final uncovered counts 0 versus 82 and 18), while tabu tenures and WalkSAT-style noise were harmful on these instances. Simulated annealing sustained a 33× higher move rate and still lost to the focused search — move quality beats move count here.

3.2 Seeding and descent

For each cell we first rebuilt a code of the incumbent size from the same construction that produced the incumbent bound (e.g., for $K_6(10, 4)$ the direct sum (1) of an optimal $K_6(4, 1)$ code of size 72 and a $K_6(6, 3)$ code of size 41, both found by direct search and verified). Descent then proceeds by remove-and-repair: delete the codeword whose private coverage (words covered by no other codeword) is smallest, and repair the deficit by local search; on success, repeat at $M-1$. Searching the small factors and re-composing (a "factor attack") was decisive for seeding; the improvements themselves come from the descent breaking out of the product structure.

3.3 Hardware and effort

All record searches ran on the 64-core Grace CPU of a single NVIDIA GH200 node (a CUDA port was implemented and verified but was latency-bound and slower than the CPU at these instance sizes). Per-cell effort ranged from seconds ($K_6(8, 4)$, $K_6(9, 5)$) to under two hours of wall clock; none of the descents had converged when stopped. Move throughput ranged from ∼1.3k moves/s per worker on the largest cells to ∼400k on the smallest.

4 Verification

False records are worse than no records, so verification is exhaustive, exact, and redundant. Every claimed code was checked by four methods, two of which share no code with the other two:

1. ball marking: mark all words within distance R of each codeword in a q^n bitmap and confirm the bitmap is full;

2. meet-in-the-middle bitset covering over a coordinate split;

3. a min-distance scan: enumerate all q^n words and compute each word's distance to the nearest codeword (this also reports the exact covering radius);

4. Hamming-graph dilation: R-fold neighborhood dilation of the code's indicator vector, computing no distances at all, with cost independent of M.

In addition, a fifth, independently written dilation verifier (produced without access to the other implementations) re-confirmed all nine codes from the published files. The largest check ($q^n = 6^{10} = 60{,}466{,}176$ words) completes in seconds.

The ancillary files include `verify_cov.py`, a standalone dependency-free Python verifier: `python3 verify_cov.py CODEFILE q n R` exhaustively re-checks any of the claimed bounds in at most a few minutes.

5 Discussion

The pattern in our experiments is clean: local search consistently beat bounds that were inherited from general constructions (relative slack between the previous upper bound and the sphere-covering lower bound of 4×–10×), and consistently failed to improve cells that had already been the object of dedicated search or algebraic construction (slack $\leq 3\times$), such as $K_3(11, 4) \leq 81$, $K_6(6, 3) \leq 41$, and $K_8(8, 4) \leq 512$. Kéri's tables contain many further cells of the first kind, for $q \in \{6, \ldots, 21\}$ and $R \geq 3$; we expect them to fall to the same method, and a systematic sweep is in progress. Since the tables have had no maintainer since 2011, we also intend to publish a merged, machine-readable bounds table (Kéri 2011 plus the post-2011 lower-bound literature plus the present improvements) alongside the code.

Data availability

The codes, the verifier, and a README are ancillary files of this arXiv submission. The full search and verification source code is public at `https://github.com/Mapika/coldcase` (covering-code track under `cov/`).

Acknowledgments

This work is AI-assisted in the strong sense: the discovery and verification pipeline was designed, implemented, and operated autonomously by Claude (Anthropic). The human author's contributions were the initiating idea (applying modern large-scale search hardware to record tables frozen in the 2000s) and the computational resources.

References

[1] G. Cohen, I. Honkala, S. Litsyn, A. Lobstein, *Covering Codes*, North-Holland, Amsterdam, 1997.

[2] G. Kéri, Tables for bounds on covering codes, `https://old.sztaki.hu/~keri/codes/` (last updated November 2011).

[3] D. Gijswijt, S. Polak, Semidefinite lower bounds for covering codes, arXiv:2504.01932, 2025.

[4] Y. Wu, C. Chen, Improved lower bounds on the domination number of hypercubes and binary codes with covering radius one, Discrete Mathematics 347 (2024) 113752.

[5] P. J. M. van Laarhoven, E. H. L. Aarts, J. H. van Lint, L. T. Wille, New upper bounds for the football pool problem for 6, 7 and 8 matches, J. Combin. Theory Ser. A 52 (1989) 304–312.

[6] P. R. J. Östergård, Constructing covering codes by tabu search, J. Combin. Des. 5 (1997) 71–80.

[7] G. Kéri, P. R. J. Östergård, Further results on the covering radius of small codes, Discrete Mathematics 307 (2007) 69–77.

[8] A. Florath, A Lean 4 formalization of q-ary covering codes with a proof-carrying database of certified bounds, arXiv:2606.09600, 2026.

A The code proving $K_6(8, 4) \leq 167$

For self-containedness we print the largest single improvement ($216 \to 167$, -23%). Each row lists codewords of $\mathbb{Z}_6^8$; the machine-readable version is the ancillary file `K6_8_4_M167.txt`.

```
42030014  50143021  30440244  25403541
40250502  02441030  35400150  50023554
50225151  12333122  33515545  44121521
14212440  42314041  35042002  34355041
05025020  12253354  05133334  42335434
03201411  35343410  24143214  54542204
32151400  10440535  43332055  32200053
30051122  00045454  52024303  50551345
15230201  24410415  54434031  23142140
33254504  25214034  21244445  53231213
04130543  01302114  34325315  50135422
45510313  51220320  45504535  22125553
31533321  55255112  04041551  42150315
02322452  15142345  02303401  04013213
33310230  11101302  41241312  25550524
13423003  22044222  20311131  14551115
02105344  10310252  10344333  42540351
22520011  15221421  51124410  24221205
11152242  01253131  04202301  53440342
11031550  20131443  30014311  22432502
05111025  41054323  33130421  00504200
20532144  41122233  34544402  40202220
21401320  23553251  51540443  55412301
53051015  32234252  12455213  12000145
31105501  01212555  40424455  13501210
52103235  35321243  24030300  24100153
05431103  33343524  31114105  02214523
45311104  40401434  53352125  01424040
21352033  30102012  03250042  52511554
43120104  43112350  20205330  34452453
54355500  04545025  23535224  44405124
11015404  50015125  50313003  01310221
```

```
41321232  00553430  25032211  51342511
14115532  52513140  55154250  43054444
23323342  31441054  33022530  02422344
12502423  34233035  00434510  41505000
30521513  05524132  04301325  21413452
15533052  44344100  14052131  40033105
25355405  11335355  41035241  13444111
14324024  23004353  03455235  33245133
44453250  54304042  43512332  13003512
31413225  55000434  45243543
```